Software Testing, Quality Assurance & Maintenance—Lecture 23

Patrick Lam
University of Waterloo

March 30, 2026

Yet More Dafny Motivation

Earlier: talked about verifying Cedar (authorization library).

Recently: using Dafny to verify toy examples.

Today: more applications of Dafny for production software.

Source

“On the Impact of Formal Verification on Software Development.”
by Eric Mugnier, Yuanyuan Zhou, Ranjit Jhala, and Michael Coblenz.
Appeared in OOPSLA 2025.

Aside: evaluating papers

Is a paper worth your time?

Check the title and abstract.
Abstract: author-provided summary.

On the Impact of Formal Verification on Software Development

Auto-active verifiers like Dafny aim to make formal methods accessible to non-expert users through SMT automation. However, despite the automation and other programmer-friendly features, they remain sparsely used in real-world software development, due to the significant effort required to apply them in practice. We interviewed 14 experienced Dafny users about their experiences using it in large-scale projects. We apply grounded theory to analyze the interviews to systematically identify how auto-active verification impacts software development, and to identify opportunities to simplify the use, and hence, expand the adoption of verification in software development.

Goals of this Lecture

- know advantages & disadvantages of Dafny for real systems;
- identify when Dafny could help;
- know best practices in deploying Dafny.

Context: Dafny Applications

So far, seen Dafny to:

- verify in the small;
- verify larger systems like Cedar.

Paper also cites other cases: cloud controllers, security modules.

Context: Auto-active Verifiers

Dafny is called an auto-active verifier.

Usually, it proves what you need, but you may have to help it.
Besides invariants, also may need assertions and lemmas.

Fully interactive: Lean and Rocq
(beyond the scope of this course).

Interactive theorem provers

Basically, give them a set of steps to prove a theorem.

Need expertise to use them.
(Much more powerful than George; can specify tactics.)

Contrast to Dafny, which undergrads can use.
But, for large codebases, Dafny is subject to automation failure.

Summary of Results

- two types of developers: *formal-first* vs *engineering-first*
- design/implementation: verification adds specification writing to design phase, and specification-proving to implementation phase.

  also in implementation: debugging failed proof attempts, hardening the code for more robust verification.
- testing/deployment: verification affects testing & code reviews, making reviewing harder.

Methodology

- 14 participants, interviewed by authors (semi-structured)
- grad students, computer scientists, software engineers, professors; experience from $<$ 1 year to $\geq$ 10 years.

- projects: less than 1000 LOC to more than 10kLOC
- domains: distributed systems, compilers, mathematical proof, security critical, cloud, authorization, business critical, file system, storage system, cryptography, blockchain

Impact of Verification on Development

- *specification* now needs formal specifications;
- *implementation* now needs proofs;
- *debugging* includes proof debugging;
- *hardening* makes proofs more resilient to change.

Specification

How do specifications get written?

When: some started with the spec; some started with an impl and test on real data; some write concurrently.

What: we talked about the oracle problem; often use the spec as an oracle.

Where's the specification from?

documentation (including design documents); tests; code.

From docs: informal spec first, then Dafny contracts.

but: "Documentation is often incomplete and vague".

Where's the specification from?

documentation (including design documents); tests; code.

Tests: generalize into specifications (can be challenging).

Code: ought to embody the specification, but extracting is not obvious, and must omit implementation details.

Specification styles

`method` or `function`?

Functional specs in 5 of the interviews:
no need to track what state has changed;
aliasing; complex control-flow.

Previous lecture: `swap`, definitely imperative.
Does in-place modification of state; nothing
else changes.

Functional `swap`: return a copy of the array.

Figure 3 example

```
method UpperBoundedAdditionImpl(x: uint64, y: uint64, u: uint64)
        returns (sum: uint64)
  ensures sum as int = UpperBoundedAddition(x as int, y as int,
        UpperBoundFinite(u as int));
{
  if y ≥ u {
    sum := u; // write
  } else if x ≥ u - y {
    sum := u; // write
  } else {
    sum := x + y; // write
  }
}

// functional spec: no intermediate state
function UpperBoundedAddition(x: int, y: int, u: UpperBound): int
{
  // LeqUpperBound checks if a given integer x
  // is less than or equal to the specified upper bound u.
  if LeqUpperBound(x + y, u) then x + y else u.n
}
```

Problem: aliasing

```
/** A helper method to swap two positions in an array */
method swap(a: array<int>, pos1: nat, pos2: nat)
```

OK, array `a` changes, but are there any aliases of it that simultaenously change?

Rust: uniqueness, enforced.
C++: uniqueness is an option.

Logical vs implementation types

One participant says:

spec should use logical types (sequences, maps) and not ipmlementation types (vectors, hash maps).

Also: we haven't seen a layer that "hides the details of the algorithm and exposes an interface".

Use a `predicate` to summarize what to prove.

Getting satisfaction

11/14 participants would only be satisfied by a proof.

other 3: sometimes trust their impls;
2 skipped “well-tested or relatively straight forward functions”.

Writing proofs

As you know: writing proofs is hard.
Proofs can be 10$\times$ lines of implementation.

Some only committed verified code to repo; others committed proofs developed gradually with assumptions.

All developed proofs incrementally with assumptions (beware).

Debugging proofs

For code: reproduce problem, figure out how to fix it.

For proof: like “prove or disprove” math problem.
Is the property actually true?
Use testing to help.

Debugging experiences

What's a bug?

Here: specification doesn't match the implementation.
Happens often.

Dafny can fail to prove statements without hints.
1 participant reported Dafny unsoundness.

Complaints & Workarounds

- Dafny error messages don't say why;
- used assertions to figure out what Dafny knows (like breakpoints);
- used `assume` and conditionals to understand what Dafny knows.

Wider challenge: "build a mental model of implicit verifier state".
Interactive verifiers easier to use in that sense; for Dafny, annotations can help.

Proof Hardening

Kind of like code refactoring to eliminate smells.

Proof works now, but might not work on another machine, another solver version, or after small impl change.

some little fiddly change to the code, just maybe pass one more element in the state [. . .] and all the proofs would just stop working.

“soul crushing”; induces “existential dread”

Brittleness

3 participants had not encountered brittle proofs.

Some addressed brittleness reactively: increase resource limits.
Others proactively: month-long major refactoring; or, fix timeouts before continuing.

Why brittleness?

Dafny doesn't know what's important.
Passes too much context to server, but sometimes the wrong context.

Try to make state visible, but gigabytes of state is unreadable.

Frame conditions also get large.

Dealing with brittleness

Many: monitor solver resource count to see if things getting hard.

Also: add or remove hint assertions, reducing solving time.

Other tools:

- "unsat core"
- `assert false`
- using smaller lemmas;
- `opaque` and `reveal`

Dealing with brittleness II

More systematically: 4 used structured approaches, including style guides, to reduce automation.

Report: Dafny is getting better.

Brittleness only matters to you if you observe it.

Impact of Verification on Deployment

Yes, we'll talk about software quality assurance and maintenance.

Also, deployment of verified software.

How Testing

Everyone does it.

Half compare spec to impl or to expected behaviour.

- *verified specifications*: some just wanted executable model of the code; not actually deployed, but model compared to impl.
- *verified implementations*: verified code that would be deployed, use testing to check vs unverified legacy code.

Differential random testing applicable.

Additionally: some verified new implementations of new specifications; there was no legacy.

Unit versus end-to-end testing

e.g. JUnit used all the time for system tests.

Participants did both unit testing and end-to-end testing, though what infrastructure is unclear.

Real Data

One participant ran verified impl in parallel with prod, finding differences, even after fuzzing.

Another participant said people wanted to see verified code working on real data.

NB: Dafny can be transpiled to other languages & tested for performance.

Code Review

Is a best practice; all multi-author projects here used it.

But: makes reviews larger. Need to review specs & transpiled code, plus impls.
Can be “quite large”, and proof changes intertwined with code.

Participants focussed on specifications, plus non-verified code (e.g. libraries).

What to Review

One participant reviewed proofs for clarity & maintainability.

Sometimes participants reviewed transpiled code (due to lack of trust), but not all the time.

Especially likely when other teams would interact with transpiled code.

Impact of Verification on Maintenance

Testing: confidently refactor & optimize code. (Rust: “fearless concurrency” due to type system).

Verification: even better; can rely on verified specification even more than on your test suite; Dafny will tell you of divergence.

A change will often require both spec & proof changes.
But, get more knowledge of the system as a result: what might break?

Breaking changes

Can see which spec no longer holds, and focus testing accordingly.

Use small fixes for spec and proof.

And, get some confidence that a change is not breaking if Dafny agrees.

All this relies on verified code being deployed prior to change.

Optimization

Participants reported being more willing to make changes due to verification.

e.g. localized changes that do not affect the specification, such as algorithmic optimization and removing redundant checks.

Verification implies that new code continues to meet the specification.

(Redundant checks: those that can be shown at proof time, eg with `requires`).

Summary

Yes, people use Dafny on real systems, perhaps via transpilers.

Affects all stages of software developments. Results in software that meets its formally-expressed specifications, and enables lower-risk changes.